



EF QueryLens

See the SQL behind every LINQ expression — right in your editor.

Visual Studio Marketplace **v1.0.17** Rider **v1.0.17** License **MIT**



EF QueryLens translates EF Core LINQ to SQL at hover time. A shared daemon does the work — no database connection needed.

VS Code  Rider Visual Studio

Get started in four steps

1. Install the extension for your IDE
2. Run "EF QueryLens: Setup" to generate your offline DbContext factory
3. Build your solution
4. Hover any LINQ query → SQL appears

That's it. Setup scans your `AddDbContext` registrations and auto-generates everything it needs.

Features

 View SQL on Hover Hover any EF Core LINQ expression to see the generated SQL inline. No context switching, no logging, no guessing.	 Open SQL Panel Open the full SQL in a dedicated editor panel. Copy to clipboard or inspect split-query statements with per-split labels.
 Auto Setup Run EF QueryLens: Setup once. QueryLens scans your <code>AddDbContext</code> registrations, generates a gitignored factory, and is ready to go — no manual wiring.	 Works Offline SQL is generated from your compiled assembly, not a live database. Works anywhere your code builds — no connection string required at runtime.
 Every Major .NET IDE One daemon, consistent results across VS Code , JetBrains Rider , and Visual Studio .	 Provider-Aware SQL Server, PostgreSQL, and MySQL are each formatted with their correct dialect — what you see is what EF Core actually sends.
 MCP Server Expose LINQ-to-SQL translation to AI agents and automation via a built-in MCP server. Query your data model programmatically.	 Split-Query Support Multi-statement split queries are rendered with per-statement labels so you see exactly how EF Core breaks up the load.

Setup

1. Install the extension

IDE	Link
VS Code	Visual Studio Marketplace
JetBrains Rider	JetBrains Marketplace
Visual Studio	Visual Studio Marketplace

2. Set up your offline DbContext factory

EF QueryLens needs a lightweight offline DbContext to generate SQL without a live database. Run **EF QueryLens: Setup** from the command palette — QueryLens inspects your `AddDbContext` registrations and generates the factory for you, gitignored and ready to go.

You can also trigger setup by hovering any LINQ query and clicking **Set up QueryLens for this project**.

For multiple DbContexts, custom options, or manual factory authoring, see [Factory Setup](#).

3. Build your solution

```
dotnet build
```

This compiles the generated factory so QueryLens can load it.

4. Hover any LINQ expression

SQL appears in the hover popup. Use the inline actions to **copy**, **open** the full SQL panel, or **refresh**.

Works everywhere

► IDE screenshots

Documentation

- [Getting Started](#)
- [Factory Setup](#)
- [IDE Support](#)
- [Providers](#)
- [MCP Server](#)
- [CLI Reference](#)
- [Architecture](#)

Build & Test

► Build commands

Contributing

See [CONTRIBUTING.md](#).

Security

See [SECURITY.md](#).

License

MIT — see [LICENSE](#).